

EV Range & Driving Efficiency Analytics

AWS setup guide: Amazon S3 → AWS Glue → Glue Data Catalog → Amazon Athena

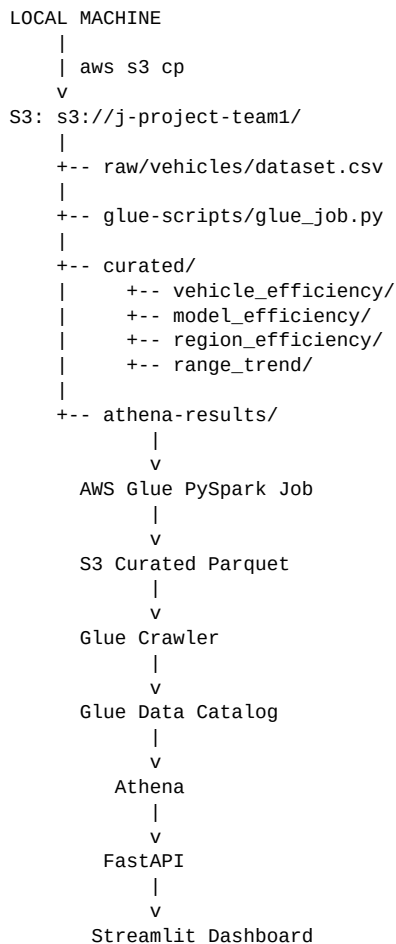
This guide gives two ways to build the AWS data pipeline: (1) AWS Management Console/manual configuration and (2) AWS CLI commands. It is written for the EV analytics project and uses an S3 bucket named **j-project-team1** as the working example. Replace it if you use a different bucket.

NOTE

AWS console labels can change over time. The steps below follow the current AWS service model. AWS Glue crawlers infer schema and register tables in the Glue Data Catalog, and Athena uses the Glue Data Catalog metadata to query data stored in S3. [cite](#) [turn0search5](#) [turn0search1](#)

Security: do not put AWS access keys or secret keys in your project or PDF. For local CLI access, use **aws configure** or an IAM Identity Center/profile. For Glue, use an IAM service role.

1. Final architecture



The important connection is: **S3 stores the files; Glue Crawler discovers the schema and creates Catalog tables; Athena uses those Catalog tables to query the S3 data.** AWS documents this integration explicitly.

[cite](#) [turn0search1](#) [turn0search5](#)

2. Prerequisites

AWS CLI
Python 3.x

Git
Your EV project ZIP
An AWS account
An IAM identity allowed to create/use S3, Glue, IAM and Athena resources

Configure AWS CLI

```
aws --version
aws configure
aws sts get-caller-identity
```

When prompted by **aws configure**, select your AWS Region, for example **ap-south-1**. Keep credentials out of the repository.

3. S3 bucket

Manual configuration — AWS Console

1. Open **Amazon S3** → **General purpose buckets**.
2. Create a bucket if you do not already have one.
3. Example: **j-project-team1**.
4. Keep Block Public Access enabled.
5. Use the region you intend to use for the project.

CLI

```
# Create a bucket in ap-south-1
aws s3api create-bucket --bucket j-project-team1 --region ap-south-1 --create-bucket-configuration LocationConstraint=ap-south-1

# Verify
aws s3 ls
aws s3api head-bucket --bucket j-project-team1
```

NOTE

If the bucket already exists, do not run create-bucket again. You can simply use it.

Create project prefixes

```
aws s3api put-object --bucket j-project-team1 --key raw/vehicles/
aws s3api put-object --bucket j-project-team1 --key glue-scripts/
aws s3api put-object --bucket j-project-team1 --key curated/
aws s3api put-object --bucket j-project-team1 --key athena-results/
```

S3 prefixes are logical folders; uploading the first object under a prefix is enough to make it visible in the console.

4. Upload dataset.csv

Manual configuration

Open S3 → **j-project-team1** → **raw** → **vehicles** → Upload → choose **data/dataset.csv** → Upload.

CLI

```
aws s3 cp data/dataset.csv s3://j-project-team1/raw/vehicles/dataset.csv
aws s3 ls s3://j-project-team1/raw/vehicles/
```

Expected object:

```
s3://j-project-team1/raw/vehicles/dataset.csv
```

5. IAM role for AWS Glue

Glue jobs and crawlers assume an IAM role. AWS states that the role needs permissions to access the S3 sources, targets, scripts and temporary directories. For S3 sources, Glue needs permissions such as ListBucket/GetObject; targets additionally need write permissions. ■cite■turn0search0■turn0search8■

Manual configuration

1. Open **IAM** → **Roles** → **Create role**.
2. Trusted entity: **AWS service**.
3. Use case/service: **Glue**.
4. Attach **AWSGlueServiceRole**.
5. Create role named **AWSGlueServiceRole-EVAnalytics**.
6. Add a least-privilege inline S3 policy for your bucket.

Recommended S3 policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:ListBucket",
        "s3:GetBucketLocation"
      ],
      "Resource": "arn:aws:s3:::j-project-team1"
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:GetObjectVersion"
      ],
      "Resource": [
        "arn:aws:s3:::j-project-team1/raw/*",
        "arn:aws:s3:::j-project-team1/glue-scripts/*"
      ]
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject"
      ],
      "Resource": "arn:aws:s3:::j-project-team1/curated/*"
    }
  ]
}
```

AWS's Glue documentation recommends granting the Glue role S3 access appropriate to the sources and targets rather than relying on broad S3 permissions. ■cite■turn0search0■

CLI: create the Glue role

```
aws iam create-role --role-name AWSGlueServiceRole-EVAnalytics --assume-role-policy-document file://trust-policy.json
aws iam attach-role-policy --role-name AWSGlueServiceRole-EVAnalytics --policy-arn arn:aws:iam::aws:policy/service-role/AWSGlueServiceRole
aws iam put-role-policy --role-name AWSGlueServiceRole-EVAnalytics --policy-name EVAnalyticsS3Access --policy-document file://s3-policy.json
aws iam get-role --role-name AWSGlueServiceRole-EVAnalytics
```

Create **trust-policy.json** with the Glue service principal:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"Service": "glue.amazonaws.com"},
    "Action": "sts:AssumeRole"
  }]
}
```

6. Upload the Glue PySpark script

CLI

```
aws s3 cp scripts/glue_job.py s3://j-project-team1/glue-scripts/glue_job.py

aws s3 ls s3://j-project-team1/glue-scripts/
```

Manual

S3 → j-project-team1 → glue-scripts → Upload → select **scripts/glue_job.py**.

7. Create the AWS Glue ETL job

Manual configuration

Open **AWS Glue** → **ETL jobs** → **Create job**. Create a Spark ETL job. Use the IAM role **AWSGlueServiceRole-EVAnalytics**. Set the script location to:

```
s3://j-project-team1/glue-scripts/glue_job.py
```

The Glue CLI/API identifies a Spark ETL job with the **glueetl** command and accepts an S3 script location.

■cite■turn0search4■

Job parameters

```
--input_path
s3://j-project-team1/raw/vehicles/dataset.csv

--output_path
s3://j-project-team1/curated/
```

CLI create-job example

```
aws glue create-job --name ev-range-analytics-job --role AWSGlueServiceRole-EVAnalytics --command '{
  "Name": "glueetl",
  "ScriptLocation": "s3://j-project-team1/glue-scripts/glue_job.py",
  "PythonVersion": "3"
}' --glue-version "4.0" --worker-type G.1X --number-of-workers 2 --default-arguments '{
  "--job-language":"python",
  "--input_path":"s3://j-project-team1/raw/vehicles/dataset.csv",
  "--output_path":"s3://j-project-team1/curated/"
}' --region ap-south-1
```

If your AWS account offers a newer Glue version, choose the version supported by your region/account and adjust the CLI argument accordingly.

Run the job

```
aws glue start-job-run --job-name ev-range-analytics-job

aws glue get-job-runs --job-name ev-range-analytics-job
```

After a successful run, inspect S3:

```
aws s3 ls s3://j-project-team1/curated/
```

8. Expected curated S3 layout

```
s3://j-project-team1/curated/
■■■ vehicle_efficiency/
■■■ model_efficiency/
■■■ region_efficiency/
■■■ range_trend/
```

Each prefix contains Parquet objects written by the PySpark job.

9. Create Glue Data Catalog database

Manual

AWS Glue → Data Catalog → Databases → Add database → name it **ev_range_analytics**.

CLI

```
aws glue create-database --database-input '{"Name":"ev_range_analytics"}' --region ap-south-1
```

```
aws glue get-database --name ev_range_analytics --region ap-south-1
```

The Glue Data Catalog is AWS's persistent metadata store. ■cite■turn0search2■

10. Create the Glue crawler

A crawler scans an S3 location, determines the schema, and creates or updates table definitions in the Glue Data Catalog. Those tables can then be queried from Athena. ■cite■turn0search5■turn0search11■

Important for this project

Because your four curated prefixes have different schemas, use **separate crawler targets/prefixes** or separate crawlers. Do not point one crawler only at **curated/** and assume it will always create the four desired tables. AWS documents that separate S3 paths should be used when different table schemas need separate tables.

■cite■turn0search13■

Recommended crawler setup

Crawler 1:
s3://j-project-team1/curated/vehicle_efficiency/

Crawler 2:
s3://j-project-team1/curated/model_efficiency/

Crawler 3:
s3://j-project-team1/curated/region_efficiency/

Crawler 4:
s3://j-project-team1/curated/range_trend/

Manual crawler

AWS Glue → Crawlers → Create crawler → name **ev-vehicle-efficiency-crawler** → S3 data source → S3 path **s3://j-project-team1/curated/vehicle_efficiency/** → choose Glue IAM role → target database **ev_range_analytics** → create → Run crawler.

Repeat for the other three prefixes. AWS's current crawler workflow includes crawler properties, data sources, security settings, output/scheduling and review/create. ■cite■turn0search11■

CLI example

```
aws glue create-crawler --name ev-vehicle-efficiency-crawler --role arn:aws:iam::YOUR_ACCOUNT_ID:role/AWSGlueServiceRole-B
  "S3Targets": [
    {
      "Path": "s3://j-project-team1/curated/vehicle_efficiency/"
    }
  ]
}' --region ap-south-1
```

```
aws glue start-crawler --name ev-vehicle-efficiency-crawler --region ap-south-1
```

```
aws glue get-crawler --name ev-vehicle-efficiency-crawler --region ap-south-1
```

Repeat the command with the other three S3 prefixes and crawler names.

11. Verify Glue Catalog tables

```
aws glue get-tables --database-name ev_range_analytics --region ap-south-1
```

You want tables similar to:

```
vehicle_efficiency
model_efficiency
region_efficiency
range_trend
```

12. Configure Amazon Athena

Manual

Open **Amazon Athena** → **Query editor**. Select data source **AwsDataCatalog** and database **ev_range_analytics**. Athena's S3 + Glue Data Catalog integration uses the Catalog for table/column metadata and the underlying S3 objects for the actual data. ■cite■turn0search1■turn0search10■

Set the Athena query result location to:

```
s3://j-project-team1/athena-results/
```

NOTE

Athena query results are written to S3. Keep the results prefix separate from the source/curated prefixes.

CLI

Athena is commonly configured through a workgroup. You can create one with a result location:

```
aws athena create-work-group --name ev-analytics-workgroup --configuration '{
  "ResultConfiguration": {
    "OutputLocation": "s3://j-project-team1/athena-results/"
  }
}' --region ap-south-1
```

Then use that workgroup when starting queries.

13. First Athena queries

List tables

```
SHOW TABLES;
```

Inspect vehicle table

```
SELECT *
FROM vehicle_efficiency
LIMIT 10;
```

Top 5 vehicles

```
SELECT
  vehicle_id,
  model,
  region,
  total_distance_km,
  overall_efficiency
FROM vehicle_efficiency
WHERE overall_efficiency IS NOT NULL
ORDER BY overall_efficiency DESC
LIMIT 5;
```

Bottom 5 vehicles

```
SELECT
  vehicle_id,
  model,
  region,
  total_distance_km,
```

```

    overall_efficiency
FROM vehicle_efficiency
WHERE overall_efficiency IS NOT NULL
ORDER BY overall_efficiency ASC
LIMIT 5;

```

Model efficiency

```

SELECT
    model,
    total_distance_km,
    total_battery_consumed,
    overall_efficiency
FROM model_efficiency
ORDER BY overall_efficiency DESC;

```

Region efficiency

```

SELECT
    region,
    total_distance_km,
    total_battery_consumed,
    overall_efficiency
FROM region_efficiency
ORDER BY overall_efficiency DESC;

```

Range trend

```

SELECT
    date,
    avg_estimated_range_km,
    avg_battery_percent
FROM range_trend
ORDER BY date;

```

14. CLI: run an Athena query

```
aws athena start-query-execution --query-string "SELECT * FROM vehicle_efficiency LIMIT 10" --query-execution-context Data
```

The command returns a QueryExecutionId. Check it with:

```
aws athena get-query-execution --query-execution-id YOUR_QUERY_EXECUTION_ID --region ap-south-1
```

```
aws athena get-query-results --query-execution-id YOUR_QUERY_EXECUTION_ID --region ap-south-1
```

15. How S3, Glue and Athena are actually connected

1. S3
Stores:
dataset.csv
and curated Parquet
2. Glue Job
Reads:
s3://.../raw/vehicles/dataset.csv

Writes:
s3://.../curated/...
3. Glue Crawler
Reads schema from:
s3://.../curated/vehicle_efficiency/

Creates metadata:
Glue Catalog table vehicle_efficiency
4. Athena
Reads table metadata:
Glue Catalog

Reads actual data:
S3 Parquet
5. FastAPI
Sends SQL to Athena

```
6. Streamlit
  Calls FastAPI and displays:
  Top 5
  Bottom 5
  Model efficiency
  Region efficiency
  Range trend
```

There is no direct JDBC-style 'connection' from Athena to S3 that you manually configure. Athena uses the Glue Data Catalog metadata to locate and interpret the S3 data. ■cite■turn0search1■turn0search12■

16. Troubleshooting checklist

Glue says AccessDenied

Check the Glue IAM role. It must be trusted by Glue and have S3 permissions for the exact raw, curated and script prefixes. AWS documents these source/target permission requirements. ■cite■turn0search0■

Crawler says S3 target not found

```
aws s3 ls s3://j-project-team1/curated/vehicle_efficiency/
```

The prefix must contain actual Parquet files before the crawler can discover the table.

Athena says table not found

Check that the crawler succeeded and that the table exists in database **ev_range_analytics**:

```
aws glue get-tables --database-name ev_range_analytics --region ap-south-1
```

Athena returns zero rows

Check the table's S3 Location in Glue. Make sure it points to the correct curated prefix and that the Parquet files exist.

Athena query fails because of output location

Configure a result location such as:

```
s3://j-project-team1/athena-results/
```

Glue crawler creates the wrong number of tables

Use separate crawler targets for each differently structured curated prefix. AWS warns that different schemas under the same S3 prefix can cause catalog/table interpretation problems. ■cite■turn0search13■

17. Final validation checklist

#	Check	Expected result
1	AWS CLI	aws sts get-caller-identity works
2	S3 raw	dataset.csv exists under raw/vehicles/
3	Glue IAM	Glue role can read raw and write curated
4	Glue script	glue_job.py exists in glue-scripts/
5	Glue job	Job run succeeds
6	Curated S3	Four Parquet prefixes exist
7	Glue DB	ev_range_analytics exists
8	Crawler	Crawler succeeds

#	Check	Expected result
9	Catalog	Four tables exist
10	Athena	SELECT query returns rows
11	Top 5	Top efficiency vehicles returned
12	Bottom 5	Bottom efficiency vehicles returned

18. Project-specific values to replace

AWS_REGION = ap-south-1
S3_BUCKET_NAME = j-project-team1

RAW:
s3://j-project-team1/raw/vehicles/dataset.csv

GLUE SCRIPT:
s3://j-project-team1/glue-scripts/glue_job.py

CURATED:
s3://j-project-team1/curated/vehicle_efficiency/
s3://j-project-team1/curated/model_efficiency/
s3://j-project-team1/curated/region_efficiency/
s3://j-project-team1/curated/range_trend/

ATHENA RESULTS:
s3://j-project-team1/athena-results/

GLUE DATABASE:
ev_range_analytics

GLUE ROLE:
AWSGlueServiceRole-EVAnalytics

GLUE JOB:
ev-range-analytics-job

19. Next layer: FastAPI → Athena → Streamlit

Once the Athena queries work in the console, connect your Python backend. The FastAPI application should call Athena using boto3, wait for the query to complete, parse the results, and return JSON. Streamlit should call FastAPI rather than directly embedding AWS credentials.

```

Streamlit
  |
  | HTTP
  v
FastAPI
  |
  | boto3 / Athena API
  v
Athena
  |
  v
Glue Data Catalog
  |
  v
S3 Parquet

```

20. Official AWS references

AWS Glue IAM role requirements: AWS documents the required Glue trust relationship and S3 source/target permissions. [■cite■turn0search0■](#)

AWS Glue Data Catalog: AWS documents the Catalog as the persistent metadata store for databases and tables. [■cite■turn0search2■](#)

Glue crawlers: AWS documents how crawlers discover S3 schemas and register tables for Athena.

■cite■turn0search5■turn0search11■

Athena + Glue Catalog: Athena uses Glue Catalog metadata for S3 data. ■cite■turn0search1■